

핵심 철학

수업에서 반복적으로 강조한 원칙 두 가지가 이 프로젝트의 평가 기준입니다.

첫째, **"Why → What → How"** 순서를 지켜라. 기술 구현(How)에 매몰되지 말고, 비즈니스 가치(Why)에서 출발하여 요구사항(What)을 도출하고, 그 다음에 아키텍처(How)를 결정하라.

둘째, 모든 설계 결정에 **"왜?"**를 답할 수 있어야 한다. "정답은 없다. 오직 트레이드오프만 있을 뿐이다." 어떤 패턴을 선택했든, 그 선택의 근거와 대가를 설명할 수 있으면 좋은 설계이다.

최종 산출물은 **"이 문서 세트를 AI에게 넘기면 추가 질문 없이 구현을 시작할 수 있는가?"**를 기준으로 판단합니다. 장황한 서술 문서 없이, 구조화된 명세 자체가 곧 설계 문서입니다.

전체 설계 프로세스와 산출물

단계	수업 연결	핵심 질문	산출물
① 벤치마킹	Why에서 출발 (04강)	기존 솔루션은 왜 그렇게 만들었나?	비교표
② 페르소나 & CJM	사용자 여정 설계 (05강)	누가, 어떤 맥락에서, 왜 쓰는가?	페르소나 카드 + 여정 지도
③ 유저 스토리	요구사항 분석 (04강) + 유스케이스 (05강)	무엇을 만들 것인가?	유저 스토리 목록 (인수 조건 포함)
④ 유스케이스 명세	기본/확장/예외 흐름 (05강)	사용자와 시스템이 어떻게 대화하는가?	유스케이스 명세서
⑤ ERD	개념 모델링, 정규화 (04강)	데이터는 어떻게 생겼나?	ERD + 데이터 사전
⑥ 아키텍처 결정	아키텍처 패턴 (03강) + 트레이드오프 (01강)	시스템은 어떻게 나뉘고, 왜 그렇게 나뉘나?	아키텍처 다이어그램 + ADR
⑦ API 설계	외부 설계 (01강)	프론트와 백은 어떤 계약을 맺나?	API 명세
⑧ UI 프로토타입	와이어프레임/스토리 보드 (05강)	사용자는 무엇을 보고 어떻게 행동하나?	인터랙티브 프로토타입

1차 발표 (1주차) — "누구를 위해, 왜, 무엇을"

산출물 A. 벤치마킹 비교표

수업에서 강조한 "현상(Symptom)이 아닌 근본 원인(Root Cause)"을 보는 눈으로 기존 솔루션을 분석합니다. 단순 기능 나열이 아니라, "왜 이 기능이 이렇게 설계되었는가"까지 파악해야 합니다.

예시:

기능 항목	하이웍스	다우오피스	네이버웍스	시프티	우리 설계 (+ 근거)
출퇴근 인증	GPS, IP	GPS, Wi-Fi	-	GPS, Wi-Fi, 비콘	GPS + QR → 이유: 오프라인 환경 대응
...	...	...	...	...	...

"우리 설계" 열의 근거가 핵심입니다. 이것이 Why → What 번역의 시작점입니다.

산출물 B. 페르소나 카드 (2~3명) + 고객 여정 지도(CJM)

수업 05강에서 배운 페르소나 5가지 구성 요소를 반드시 포함합니다. "모든 사람을 만족시키려는 제품은 누구도 만족시키지 못한다."

[페르소나 1] 김지현 (28세) / 스타트업 인사담당자 / Primary

프로필: 직원 30명 규모 회사, 1인 인사팀, PC 업무 중심
목표: "매달 수작업으로 근태 정산하는 3일을 반나절로 줄이고 싶다"
고충: 엑셀 관리 → 누락 빈번, 연차 잔여일수 문의 반복 대응
기술 숙련도: 중상 (SaaS 도구 다수 사용 경험)
제약: 예산 제한으로 고가 솔루션 도입 불가

→ 설계 시사점: 대시보드 한 화면에서 현황 파악, 셀프서비스 잔여 연차 조회

[페르소나 2] 박동수 (45세) / 제조업 팀장 / Primary

프로필: 직원 15명 관리, 모바일 중심 (현장 근무)

목표: "팀원 휴가 신청을 현장에서 바로 승인하고 싶다"

고충: PC 접근 어려움, 결재 지연으로 팀원 불만

기술 숙련도: 중하 (카카오톡 수준)

제약: 작은 화면, 한 손 조작, 불안정한 네트워크

→ 설계 시사점: 모바일 최적화, 단순한 승인 UI, 오프라인 큐잉

수업에서 배운 **4가지 축(단계, 접점, 감정 곡선, 보조 지표)**을 사용합니다. 감정 곡선이 급락하는 **페인 포인트**가 곧 우리 설계의 핵심 과제입니다.

[페르소나 1: 김지현의 "월말 근태 정산" 여정]

단계: 출근기록 확인 → 이상 내역 파악 → 담당자 소명 요청 → 수정 → 정산 완료

접점: 엑셀 파일 → 육안 대조 → 메신저 문의 → 수기 수정 → 급여 시스템

감정: 😐 보통 → 😞 짜증 → 😡 최저 → 😓 반복 → 😊 해소

지표: - → 오류 발견율 → 응답 대기 시간 → 수정 횟수 → 총 소요 시간

🌟 **Pain Point:** "이상 내역 파악 → 소명 요청" 구간

→ 수작업 대조에 하루, 메신저 확인 대기에 하루 소요

→ 해결 전략: 자동 이상 감지 + 시스템 내 소명 요청/응답 기능

산출물 C. 유저 스토리 목록

수업 04강의 **사용자 스토리 형식**과 **수용 기준(Acceptance Criteria)**을 따릅니다. 수업에서 강조한 **안티패턴(누락, 모순, 모호)**을 피하려면 **인수 조건**이 핵심입니다.

[US-001] 로그인

- As a 사용자

- I want to 이메일과 비밀번호로 로그인

- So that 그룹웨어 기능에 접근할 수 있다

- 우선순위: Must

- 인수 조건:

· 이메일 형식 아니면 "이메일 형식을 확인하세요" 즉시 표시 (실시간 Validation)

- 5회 연속 실패 시 30분 잠금 + 잠금 해제 안내 메일 발송
- 로그인 성공 시 JWT 발급 (Access 30분 / Refresh 7일)
- 비밀번호는 bcrypt 해시 저장

- [US-007] 휴가 신청
 - As a 직원
 - I want to 휴가 유형, 날짜, 사유를 입력하여 휴가를 신청
 - So that 결재자의 승인을 받아 공식적으로 휴가를 사용할 수 있다
 - 우선순위: Must
- 인수 조건:
 - 잔여 일수 부족 시 신청 버튼 Disabled + "잔여 연차가 부족합니다" 표시
 - 신청 완료 시 결재자에게 알림 발송
 - 과거 날짜 선택 불가 (날짜 선택기 제약)
 - 반차 신청 시 오전/오후 선택 필수

- [US-007] 휴가 신청
 - As a 직원
 - I want to 휴가 유형, 날짜, 사유를 입력하여 휴가를 신청
 - So that 결재자의 승인을 받아 공식적으로 휴가를 사용할 수 있다
 - 우선순위: Must
 - 인수 조건:
 - 잔여 일수 부족 시 신청 버튼 Disabled + "잔여 연차가 부족합니다" 표시
 - 신청 완료 시 결재자에게 알림 발송
 - 과거 날짜 선택 불가 (날짜 선택기 제약)
 - 반차 신청 시 오전/오후 선택 필수

- [US-007] 휴가 신청
 - As a 직원
 - I want to 휴가 유형, 날짜, 사유를 입력하여 휴가를 신청
 - So that 결재자의 승인을 받아 공식적으로 휴가를 사용할 수 있다
 - 우선순위: Must
 - 인수 조건:
 - 잔여 일수 부족 시 신청 버튼 Disabled + "잔여 연차가 부족합니다" 표시
 - 신청 완료 시 결재자에게 알림 발송
 - 과거 날짜 선택 불가 (날짜 선택기 제약)
 - 반차 신청 시 오전/오후 선택 필수

우선순위는 수업에서 배운 Value vs Effort 매트릭스를 활용합니다:

	높은 가치	낮은 가치
적은 노력	Quick Wins (최우선)	Fill-ins (자투리)
많은 노력	Flagship (로드맵)	Time Sinks (제외)

공통 + 선택 기능 합쳐 **15~25개** 유저 스토리를 작성하되, Must / Should / Could로 분류합니다.

산출물 D. 화면 흐름도

수업 05강의 **스토리보드** 개념을 간소화한 것입니다. 유스케이스 다이어그램 대신, 실제 화면 단위의 사용자 흐름을 그립니다.

[로그인] → [대시보드] → [휴가 목록] → [휴가 신청 폼] → [확인 모달] → [휴가 목록(갱신)]
 → [출퇴근 기록]
 → [조직도] → [사용자 상세]
 → [결재함] → [결재 상세] → [승인/반려]

1차 발표 평가 주안점

수업 개념과 직접 연결하여 평가합니다:

- ① **Why → What 번역이 되었는가?** 벤치마킹에서 도출한 인사이트가 페르소나의 고충과 연결되고, 그것이 유저 스토리로 구체화되었는가.
- ② **페르소나가 살아있는가?** 수업에서 경고한 "동상이몽"을 방지할 수 있을 만큼 구체적인가. 프로필/목표/고충/기술숙련도/제약이 모두 있는가.
- ③ **인수 조건이 모호하지 않은가?** 수업의 안티패턴(모호: "빠르게 동작해야 한다")을 피하고, 측정 가능한 기준으로 작성했는가.
- ④ **범위(In/Out)가 명확한가?** YAGNI 원칙에 따라, 현재 필요한 것과 미래로 미룰 것을 구분했는가.

2차 발표 (2주차) — "어떻게 나누고, 왜 그렇게 나눴나"

산출물 E. 유스케이스 명세서 (핵심 3~5개)

수업 05강에서 배운 유스케이스 명세서 템플릿을 그대로 사용합니다. 기본 흐름(Happy Path)뿐 아니라, **대안 흐름**과 **예외 처리**까지 반드시 포함해야 합니다. 수업에서 강조한 "사전/사후 조건 + 불변식"도 빠짐없이 기술합니다.

유스케이스명: 휴가 신청하기

ID: UC-LEAVE-01

주요 액터: 직원 (Primary)

보조 액터: 알림 시스템 (Supporting)

사전 조건:

- 사용자가 시스템에 로그인한 상태
- 사용자의 잔여 연차 정보가 조회 가능한 상태

사후 조건:

- leave_requests 테이블에 status='PENDING' 레코드 생성
- 결재자에게 알림 발송 완료

불변식:

- 잔여 연차 일수는 음수가 될 수 없음
- 하나의 날짜에 중복 휴가 신청 불가

기본 흐름 (Happy Path):

1. 직원이 휴가 신청 화면으로 진입한다
2. 시스템은 잔여 연차 일수와 휴가 유형 목록을 표시한다
3. 직원이 휴가 유형(연차), 시작일, 종료일, 사유를 입력한다
4. 시스템은 입력값을 검증한다 (날짜 유효성, 중복 여부, 잔여일수)
5. 직원이 '신청' 버튼을 누른다
6. 시스템은 확인 modal("연차 2일을 신청합니다. 진행하시겠습니까?")을 표시한다
7. 직원이 '확인'을 누른다
8. 시스템은 휴가 신청을 저장하고, 결재자에게 알림을 발송한다
9. 시스템은 "신청이 완료되었습니다" 메시지와 함께 휴가 목록으로 이동한다

대안 흐름:

- A1. 반차 신청 (3단계에서 분기)
- 3a. 직원이 '반차'를 선택한다
 - 3b. 시스템은 오전/오후 선택 UI를 추가 표시한다
 - 3c. 직원이 오전/오후를 선택하고 기본 흐름 4단계로 복귀한다

A2. 신청 취소 (6단계에서 분기)

- 6a. 직원이 '취소'를 누른다
- 6b. 시스템은 입력 내용을 유지한 채 신청 폼으로 복귀한다

예외 흐름:

- E1. 잔여 일수 부족 (4단계에서 발생)
 - 4a. 시스템은 "잔여 연차가 부족합니다 (잔여: 1일, 요청: 2일)" 메시지를 표시한다
 - 4b. 신청 버튼을 Disabled 처리한다
- E2. 날짜 중복 (4단계에서 발생)
 - 4a. 시스템은 "해당 날짜에 이미 승인된 휴가가 있습니다" 메시지를 표시한다
- E3. 알림 발송 실패 (8단계에서 발생)
 - 8a. 휴가 신청 저장은 완료하되, 알림 발송을 재시도 큐에 등록한다
 - 8b. 사용자에게는 정상 완료 메시지를 표시한다 (알림 실패는 비동기 처리)

이 명세서가 있으면 AI에게 "이 유스케이스의 백엔드 API와 프론트엔드 화면을 구현해줘"라고 바로 지시할 수 있습니다.

산출물 F. ERD + 데이터 사전

수업 04강에서 배운 **개체-속성-관계 모델링**과 **정규화(1NF→2NF→3NF)** 원칙을 적용합니다.

[테이블: leave_requests] 휴가 신청			
컬럼명	타입	제약조건	설명
id	BIGINT	PK, AUTO_INCREMENT	휴가 신청 고유 ID
user_id	BIGINT	FK → users, NOT NULL	신청자
leave_type	VARCHAR(20)	NOT NULL	ANNUAL / HALF_AM / HALF_PM / TIME
start_date	DATE	NOT NULL	시작일
end_date	DATE	NOT NULL	종료일 (반차 시 start_date와 동일)
days_used	DECIMAL(3,1)	NOT NULL	사용 일수 (연차=N, 반차=0.5)
status	VARCHAR(20)	DEFAULT 'PENDING'	PENDING / APPROVED / REJECTED / CANCELLED
approver_id	BIGINT	FK → users	결재자
reason	TEXT	NULLABLE	사유
rejected_reason	TEXT	NULLABLE	반려 사유
created_at	TIMESTAMP	NOT NULL	신청 일시
updated_at	TIMESTAMP	NOT NULL	최종 수정 일시

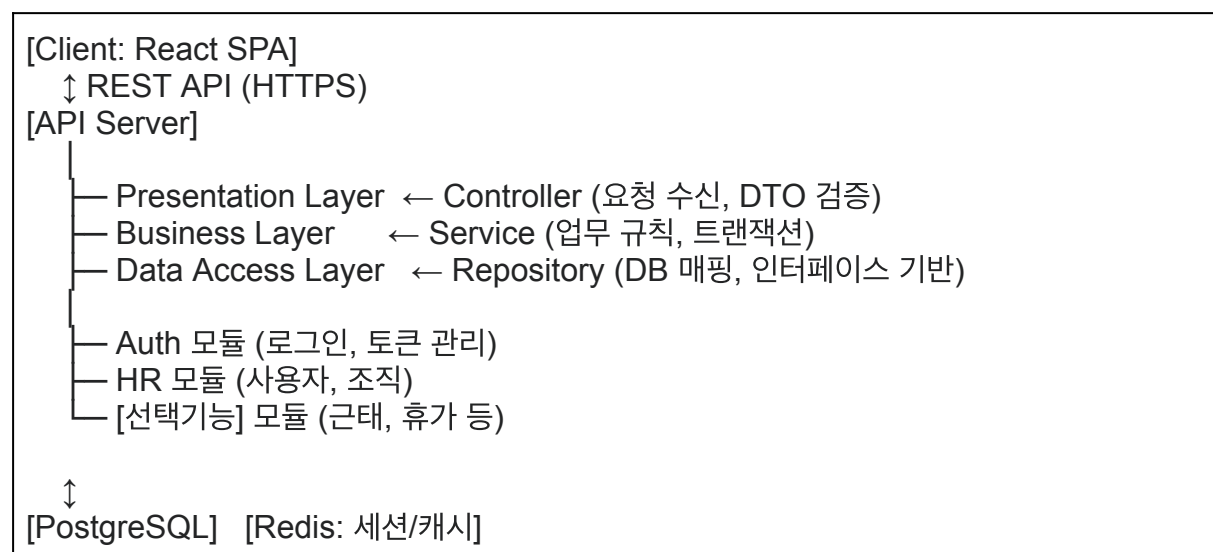
정규화 검증 포인트: 사용자 이름이나 부서명 같은 정보를 leave_requests에 중복 저장하지 않았는가? (2NF 위반 방지) 결재자의 부서 정보가 users 테이블을 통해서만 참조되는가? (3NF)

산출물 G. 아키텍처 다이어그램 + ADR

수업 03강에서 배운 **아키텍처 패턴**과 01강에서 배운 **트레이드오프 의사결정**을 실전에 적용하는 단계입니다.

아키텍처 다이어그램

수업에서 배운 패턴 중 어떤 것을 선택했는지, 그리고 수업에서 강조한 **관심사의 분리(SoC)**와 **단방향 의존성**이 다이어그램에 반영되어야 합니다.



핵심은 **모듈 간 경계와 의존 방향**입니다. 수업에서 배운 "상위→하위 단방향 의존, 인터페이스 기반 의존"이 지켜지고 있는지를 설명할 수 있어야 합니다.

ADR (Architecture Decision Record)

수업 01강에서 배운 ADR 형식을 정확히 따릅니다.

[ADR-001] 아키텍처 패턴: 계층화(Layered) 선택
- 상태: 승인됨

- 배경: 그룹웨어는 CRUD 중심의 비즈니스 로직이 주를 이루며, 팀원 4명이 병렬 개발해야 함. 복잡한 분산 처리보다 관심사의 분리와 팀 분업이 우선.
- 선택지: ① 계층화(Layered) ② MSA ③ 모듈러 모놀리스
- 결정: 계층화 패턴 (Presentation → Business → Data Access)
- 이유:
 - 수업에서 다룬 "변경의 격리 + 팀 분업 최적화 + 낮은 진입 장벽" 3가지 장점에 부합
 - 4명 규모에서 MSA는 인지 부하(Cognitive Load)가 과도함
 - 싱크홀 안티패턴은 단순 조회에 한해 캐시 도입으로 완화 예정
- 대가:
 - 모듈 간 물리적 분리가 아니므로, 패키지 경계만으로 결합도를 통제해야 함
 - 향후 트래픽 증가 시 특정 모듈만 스케일아웃하기 어려움

[ADR-002] 인증 방식: JWT 선택

- 상태: 승인됨
- 배경: 웹 + 모바일 동시 지원 필요. 수평 확장 고려.
- 선택지: ① 세션 기반 ② JWT 기반
- 결정: JWT (Access Token + Refresh Token)
- 이유: Stateless → 서버 수평 확장 용이. 모바일에서 쿠키 관리 불필요.
- 대가: 토큰 탈취 시 만료 전까지 무효화 어려움 → Redis 블랙리스트로 완화

ADR은 최소 2~3개 작성합니다. "왜 다른 대안을 버리고 이것을 선택했는가"가 핵심입니다.

산출물 H. API 명세 (핵심 엔드포인트)

수업 01강에서 다룬 **외부 설계**의 핵심 산출물입니다. "한 번 배포되면 변경하기 어렵기 때문에(Breaking Change) 신중한 설계가 필수"라는 수업 내용을 기억하세요.

[POST] /api/v1/leave-requests

- 설명: 휴가 신청 (UC-LEAVE-01 기본 흐름 8단계)
- 인증: Bearer Token (Required)
- Request Body:

```
{
  "leave_type": "ANNUAL",      // ANNUAL | HALF_AM | HALF_PM | TIME
  "start_date": "2026-05-01",
  "end_date": "2026-05-02",
  "reason": "가족 여행"       // optional, max 500자
}
```
- Response 201:

```
{
  "id": 42,
  "status": "PENDING",

```

```
"days_used": 2.0,
"approver": { "id": 7, "name": "박동수" },
"remaining_days": 10.0
}
- Response 400: { "code": "INSUFFICIENT_LEAVE", "message": "잔여 연차가
부족합니다 (잔여: 1일)" }
- Response 400: { "code": "DUPLICATE_DATE", "message": "해당 날짜에 이미
휴가가 존재합니다" }
- Response 401: { "code": "UNAUTHORIZED" }
```

API의 에러 응답 코드가 유스케이스 명세서의 예외 흐름(E1, E2)과 1:1로 매핑되어야 합니다.
이것이 수업에서 강조한 **추적성(Traceability)**입니다.

핵심 유저 스토리에 해당하는 **10~15개 API**를 명세합니다.

산출물 I. 초기 와이어프레임

수업 05강에서 배운 **와이어프레임 원칙**(구조와 배치 우선, 정보 위계 명확화)과 **방어적 설계 패턴**(재확인, 실행취소, 명확한 공지, 제약)을 반영합니다.

주요 화면 5~8개를 Lo-fi 수준으로 작성하되, 수업에서 배운 4가지 방어적 설계 중 최소 2개 이상을 적용한 부분을 표시합니다.

예시: 휴가 신청 폼에서 잔여일수 부족 시 **제약(Constraint)** — 신청 버튼 Disabled, 삭제 시 **재확인(Confirm)** — "정말 취소하시겠습니까?" 모달

2차 발표 평가 주안점

① 유스케이스 명세의 **완결성**: 기본 흐름만 있고 예외 흐름이 없으면 감점. 수업에서 "Happy Path만으로는 부족하다"를 반복 강조했음.

② 설계 **원리 적용**: 수업에서 배운 SOLID 원칙 중 최소 2가지가 아키텍처에 어떻게 반영되었는지 설명할 수 있는가. 예: "결재 로직을 Strategy 패턴으로 분리한 이유는 OCP(개방-폐쇄 원칙)를 지키기 위함이다."

③ **ADR의 트레이드오프 분석:** 수업에서 "정답은 없다. 트레이드오프만 있다"를 핵심 메시지로 다뤘음. ADR에 "대가(Consequences)"가 빠져있으면 불완전한 결정.

④ **추적성:** 유저 스토리의 인수 조건 → 유스케이스 예외 흐름 → API 에러 코드 → ERD 컬럼이 연결되는가.

⑤ **1차 피드백 반영:** 변경 사항을 명시적으로 보여줄 것.

3차 발표 — 최종 (4주차) — "완성도와 시연"

기존 산출물 정교화

2차까지의 모든 산출물을 피드백 반영하여 최종 버전으로 완성합니다. 새로운 문서를 추가하는 것이 아니라, 빈틈을 메우고 일관성을 높이는 것입니다.

특히 수업에서 강조한 설계 원리를 상세 설계에 명시적으로 적용합니다:

수업 06강의 **SOLID** 원칙과 **고응집/저결합**을 클래스/모듈 설계에 반영하고, 그 적용 근거를 설명합니다. 예를 들어:

[설계 원리 적용 사례]

1. SRP (단일 책임): LeaveService는 휴가 비즈니스 로직만 담당.
알림 발송은 NotificationService로 분리.
→ 이유: 알림 채널 변경(이메일→카카오톡)이 휴가 로직에 영향을 주지 않도록.
2. OCP (개방-폐쇄): 휴가 유형별 일수 계산을 LeaveCalculator 인터페이스로 추상화.
AnnualLeaveCalculator, HalfDayCalculator 등 구현체를 추가하면
기존 코드 수정 없이 새 유형 지원 가능.
→ 수업에서 다룬 "다중 분기(if-else) 대신 다형성" 패턴 적용.
3. DIP (의존성 역전): Service 계층은 Repository 인터페이스에 의존.
실제 DB 구현체는 생성자 주입(DI)으로 제공.
→ 테스트 시 Mock Repository 주입 가능.

산출물 J. 인터랙티브 UI 프로토타입 + 데모

Figma 등으로 제작하며, **핵심 유스케이스 3~5개를 화면 전환으로 시연**합니다.

시연 방법은 수업 05강의 **스토리보드** 개념을 따릅니다. 단순히 화면을 보여주는 것이 아니라, 유스케이스 명세서의 흐름을 따라 걸으면서 각 화면이 어떤 API를 호출하고, 어떤 ERD 테이블의 데이터를 표시하는지 연결합니다.

[시연 시나리오 예시: UC-LEAVE-01 휴가 신청]

- 화면 1: 대시보드 → "잔여 연차: 12일" 표시 (GET /api/v1/users/me/leave-balance)
화면 2: 휴가 신청 폼 → 날짜 선택, 유형 선택

화면 3: 잔여일수 부족 시 → 버튼 Disabled + 에러 메시지 (예외 E1 시연)
 화면 4: 정상 신청 → 확인 모달 (방어적 설계: 재확인)
 화면 5: 완료 → 목록에 PENDING 상태로 추가됨

→ 결재자(박동수) 시점 전환:
 화면 6: 결재함 → 새 건 알림 표시
 화면 7: 상세 → 승인 버튼 → 확인 모달
 화면 8: 승인 완료 → 신청자에게 알림 발송

예외 흐름도 반드시 시연합니다. 수업에서 "Happy Path 외에 모든 예외 상황까지 포괄하는 완결성"을 강조했습니다.

산출물 K. 추적성 체크리스트

수업 01강에서 다룬 **추적성(Traceability)** — "요구사항과 설계 요소 간의 1:1 매핑 보장"을 한눈에 검증합니다.

유저 스토리	유스케이스	ERD 테이블	API	UI 화면	설계 원리
US-001 로그인	UC-AUTH-01	users, login_logs	POST /auth/login	로그인 화면	DIP (Repository 인터페이스)
US-007 휴가 신청	UC-LEAVE-01	leave_requests, leave_balances	POST /leave-requests	휴가 신청 폼	OCP (LeaveCalculator), SRP
US-008 휴가 승인	UC-LEAVE-02	leave_requests	PATCH /leave-requests/{id}/approve	결재함	-

3차 발표 평가 주안점

① **UI 데모 ↔ 유스케이스 ↔ API ↔ ERD 연결:** 시연 중 "이 화면은 UC-LEAVE-01의 4단계이고, POST /leave-requests를 호출하며, leave_requests 테이블에 저장됩니다"라고 연결할 수 있는가.

② **설계 원리의 명시적 적용:** 수업에서 배운 SOLID, 고응집/저결합, 캡슐화 등이 구체적으로 어디에 어떻게 적용되었는지 설명할 수 있는가.

③ **방어적 설계:** 수업 05강의 4가지 패턴(재확인/실행취소/명확한 공지/제약) 중 2가지 이상이 UI에 반영되었는가.

④ **진화적 아키텍처 관점:** 수업 01강에서 다룬 "설계는 스냅샷이 아닌 흐름"의 관점에서, 현재 설계가 향후 기능 추가에 어떻게 열려있는지(OCP) 설명할 수 있는가.

⑤ **2차 피드백 반영의 가시화.**

선택 기능별 설계 시 특별히 고려할 포인트

근무 관리(근태/휴가)를 선택한 팀: 근로기준법상 근로시간(주 52시간), 연차 자동 발생 규칙, 다양한 근무 형태(교대근무, 유연근무, 재택근무) 등 법적·제도적 규칙이 도메인 로직에 큰 영향을 미칩니다. 이런 비즈니스 룰을 어떻게 구조화하여 변경에 유연하게 대응할 것인지(예: Rule Engine 패턴, Strategy 패턴 등)가 설계의 핵심 과제입니다. 또한 출퇴근 인증 방식(GPS, Wi-Fi, 비콘, QR 등)의 신뢰성과 프라이버시 간 트레이드오프를 분석해야 합니다.

전자 계약을 선택한 팀: 전자서명법 관련 법적 요건, 문서 무결성 보장(해시, 타임스탬프), 계약 상태 머신(작성 → 발송 → 서명 → 완료 → 보관), 서명 순서(순차/동시), 외부 수신자(비회원) 처리 등이 핵심입니다. 문서 템플릿 관리와 서명 필드 배치를 어떻게 설계할지, 계약 문서의 장기 보관 및 검증 가능성을 어떻게 확보할지가 아키텍처적으로 중요합니다.

전자 결재를 선택한 팀: 결재선(결재 라인)의 동적 구성, 다양한 결재 유형(결재, 합의, 참조, 전결, 대결, 후결), 양식(Form) 엔진의 유연성, 조직도 연동이 핵심입니다. 결재 상태 머신의 복잡성을 어떻게 관리할 것인지, 대량의 결재 문서를 효율적으로 조회·검색하는 구조를 어떻게 설계할 것인지 고민해야 합니다.

협업을 선택한 팀: "협업"은 범위가 넓으므로, 먼저 하위 기능(프로젝트 관리, 칸반보드, 캘린더, 파일 공유, 게시판, 위키 등)의 범위를 명확히 한정해야 합니다. 실시간 동시 편집(OT 알고리즘, CRDT 등)을 다룬다면 이는 아키텍처적으로 매우 도전적인 주제이므로, 설계 수준에서의 접근 전략을 명확히 해야 합니다. 알림(Notification) 시스템 설계도 중요합니다.

메신저를 선택한 팀: 실시간 메시지 전송(WebSocket, MQTT 등), 메시지 저장 및 동기화, 읽음 확인, 파일 전송, 그룹 채팅방 관리, 조직도 기반 연락처 연동이 핵심입니다. 메시지 전송의 신뢰성(최소 1회 전달 보장 등), 대규모 동시접속 처리, 메시지 암호화(E2E Encryption)에 대한 아키텍처적 고려가 필요합니다. 오프라인 상태에서의 메시지 큐잉과 재전송 메커니즘도 설계 포인트입니다.